Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Kostenoptimale Kraftwerkseinsatzplanung mittels gemischt-ganzzahliger Programmierung – Fallstudie Power 3000

– Abschlussprojekt 2026 –

vorgelegt am 26. April 2026

an der

Hochschule für Wirtschaft und Recht Berlin

Fachbereich Duales Studium

Operations Research

Prof. Dr. Kai Helge Becker

Antonia Schikora - 77203071970

Cedric Schuster - 77206368628

Jonas Stassiuk - 772045666863

Pascal Moroschan - 77204494011

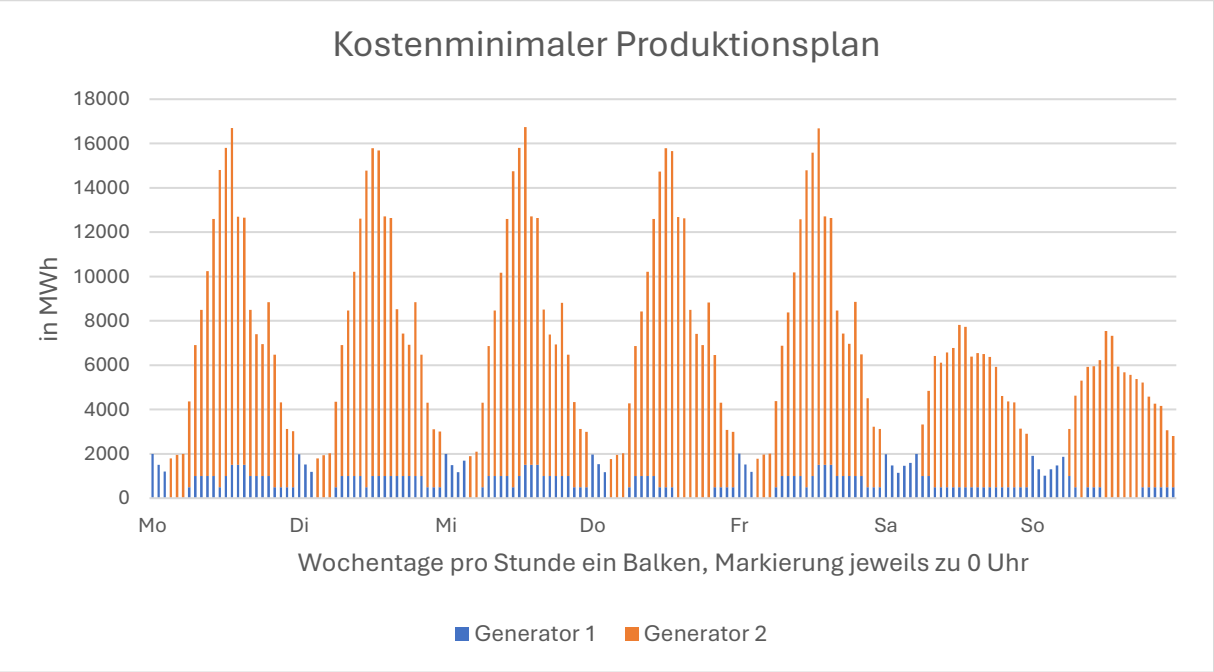
1 Executive Summary

QuantsForFuture | April 2026

Power 3000 steht vor zwei wirtschaftlichen Kernfragen: der kostenoptimalen Planung des wöchentlichen Generatoreinsatzes sowie der Rentabilität einer Kapazitätserweiterung zweier Typ-2-Generatoren. Unser Team von QuantsForFuture hat diese Fragen mittels einer computergestützten Optimierungsanalyse untersucht. Dabei wurden sämtliche betrieblichen Anforderungen, von dem prognostizierten Bedarf des betrachteten Szenarios, Last- und Kapazitätsgrenzen bis hin zu Netzstabilitätsvorgaben berücksichtigt.

Kostenminimaler Wocheneinsatz: 1,44 Mio. EUR

Für die prognostizierte Wochennachfrage ergibt sich ein kostenoptimaler Einsatzplan mit **Gesamtkosten von 1.437.135 €**. Diese Kosten setzen sich aus 133.800 € (9%) Anlasskosten, 588.600 € (41%) Fixkosten auf Minimallast und 714.735 € (50%) variablen Kosten für Produktion oberhalb der Minimallast zusammen. Der überwiegende Kostenanteil fällt auf die laufende Produktion, während die Anlasskosten nur einen geringen Kostentreiber darstellen. Dabei produzieren Generatoren des Typs 2 ca. 88% der Energie und sind für ca. 82% der Gesamtkosten verantwortlich. Eine detailliertere Betrachtung zur Ermittlung dieser Werte findet sich in Anhang C.



Upgrade von zwei Typ-2 Generatoren nicht wirtschaftlich

Das Aufwerten von zwei Typ-2 Generatoren in Bezug auf die Maximallast führt zu keiner Kostenersparnis. Die zusätzliche Kapazität wird nach der gegebenen Prognose nicht benötigt. Bei einem signifikanten Anstieg der Nachfrage ist eine erneute Prüfung sinnvoll. Weitere Informationen zur Vorgehensweise findet sich in Anhang C.

Value-Proposition

Das entwickelte Modell ermöglicht die Betrachtung weiterer Szenarien. Eingabeparameter werden aus einer Excel eingelesen, somit kann ohne technisches Wissen das aktuelle Szenario in Bezug auf Maschinenpark als auch Energienachfrage angepasst werden. Die Ergebnisse werden ebenfalls als Excel ausgegeben, wodurch eine direkte unkomplizierte Weiterarbeit ermöglicht wird. Eine tiefere Betrachtung hierfür findet sich in den Anhängen A und B.

Kosteneinsparungen durch weitere Untersuchungen

In einer ersten weiterführenden Analyse konnten weitere Einsparpotenziale entdeckt werden. Beispielsweise würde die Anschaffung eines weiteren Generators des Typen 2 zu einer wöchentlichen Kostenreduzierung von 4.000 EUR führen. QuantsForFuture empfiehlt dem Management von Power 3000 eine weitere Untersuchung in Auftrag zu geben.

Anhang A – Mathematisches Modell

Das mathematische Modell wurde so aufgestellt, dass sich damit die zwei zentralen Entscheidungsfragen für Power 3000 beantworten lassen. Dafür notwendige Überlegungen finden sich in Anhang C. Dieses Modell besteht aus: Indexmengen, Skalar- und Vektorparametern, Variablen, Nebenbedingungen und Zielfunktion.

Indexmengen

G: Generatortypen

S: Stunden des Tages

T: Wochentage. Da sich der Lastgang wöchentlich wiederholt, wird das Modell zyklisch über T formuliert, d.h. auf den letzten Tag der Woche folgt wieder der erste.

Skalarparameter

α_{oben}: Oberer Lastgangpuffer. Gibt an, um wieviel Prozent der Bedarf kurzfristig steigen darf, ohne dass neue Generatoren angelassen werden müssen.

α_{unten}: Unterer Lastgangpuffer. Gibt an, um wieviel Prozent der Bedarf kurzfristig sinken darf, ohne dass ein laufender Generator unter seine Minimallast fällt.

Vektorparameter

k_g^A: Anlasskosten je Generator vom Typ g, $g \in G$ (EUR pro Anlassung). Die Kosten fallen jedes Mal an, wenn ein Generator gestartet wird.

k_g^F: Fixkosten auf Minimallast je Generator vom Typ g, $g \in G$ (EUR pro Stunde). Diese Kosten entstehen für jede Stunde, in der ein Generator des Typs g läuft, unabhängig von der tatsächlichen Produktion.

k_g^V: Variable Kosten je erzeugter Megawattstunde (MW) oberhalb der Minimallast für Typ g, $g \in G$ (EUR pro MW und Stunde).

$p_{min,g}$: Minimallast eines Generators des Typen g, $g \in G$ (MW). Ein laufender Generator muss mindestens auf dieser Last betrieben werden.

$p_{max,g}$: Maximallast eines Generators des Typen g, $g \in G$ (MW). Ein laufender Generator darf diese Last nicht überschreiten.

n_g: Anzahl verfügbarer Generatoren vom Typ g, $g \in G$.

$d_{s,t}$: Prognostizierter Lastgang-Bedarf in Stunde s an Tag t, $s \in S$, $t \in T$ (MW).

Variablen

$x_{g,s,t}$: Gesamtproduktion aller Generatoren vom Typ g in Stunde s an Tag t, $g \in G$, $s \in S$, $t \in T$ (MW). Da das Modell nicht einzelne Generatoren, sondern Typen verfolgt, bildet diese Variable die aggregierte Leistung aller gleichzeitig laufenden Generatoren eines Typs ab.

$u_{g,s,t}$: Anzahl der laufenden Generatoren vom Typ g in Stunde s an Tag t, $g \in G$, $s \in S$, $t \in T$. Diese Variable ist ganzzahlig, da nur ganze Generatoren betrieben werden können.

$v_{g,s,t}$: Anzahl der neu gestarteten Generatoren vom Typ g in Stunde s an Tag t, $g \in G$, $s \in S$, $t \in T$. Diese Variable dient der Erfassung der Anlasskosten.

$z_{s,t}$: Binärvariable zur Modellierung der Netzwerkstabilitätsbedingung in Stunde s an Tag t, $s \in S$, $t \in T$. Der Wert 1 zeigt an, dass in dieser Stunde mehr als 3 Generatoren vom Typ 1 laufen.

Zielfunktion

Es sollen die wöchentlichen Gesamtkosten der Stromerzeugung minimiert werden. Diese setzen sich zusammen aus Anlasskosten (einmalig je Neustart), Fixkosten (stündlich je laufendem Generator auf Minimallast) sowie variablen Kosten (je produzierter Megawattstunde oberhalb der Minimallast):

$$\min \sum_{g \in G} \sum_{s \in S} \sum_{t \in T} \left[k_g^A \cdot v_{g,s,t} + k_g^F \cdot u_{g,s,t} + k_g^V \cdot \left(x_{g,s,t} - p_{min,g} \cdot u_{g,s,t} \right) \right]$$

Nebenbedingungen

<u>1) Bedarfsdeckung:</u> Zu jeder Stunde und an jedem Tag muss die Gesamtproduktion aller Generatoren mindestens dem Bedarf entsprechen.

$$\sum_{g \in G} x_{g,s,t} \geq d_{s,t} \qquad \text{für alle } s \in S,\ t \in T$$

<u>2) Produktionsminimum:</u> Ein Generator muss mindestens auf seiner Minimallast betrieben werden. Die Gesamtproduktion ist also nach unten durch die Minimallast mal die Anzahl der laufenden Generatoren (pro Typ) begrenzt.

$$x_{g,s,t} \geq p_{min,g} \cdot u_{g,s,t} \qquad \text{für alle } g \in G,\ s \in S,\ t \in T$$

3) Produktionsmaximum: Ein Generator hat eine Maximallast in MW die er pro Stunde erreichen kann. Die Gesamtproduktion ist hier also nach oben durch die Maximallast mal die Anzahl der laufenden Generatoren (pro Typ) begrenzt.

$$x_{g,s,t} \leq p_{max,g} \cdot u_{g,s,t} \qquad \text{für alle } g \in G,\ s \in S,\ t \in T$$

4) Verfügbare Generatoren: Es steht nur eine begrenzte Anzahl an Generatoren nach Typ zur Verfügung. Diese darf zu keinem Zeitpunkt überschritten werden. Die Anzahl der laufenden Generatoren des Typen muss also immer gleich oder geringer der Maximalanzahl sein.

$$u_{g,s,t} \leq n_g \qquad \text{für alle } g \in G,\ s \in S,\ t \in T$$

5) Oberer Lastgangpuffer: Zu jedem Zeitpunkt muss die Gesamtkapazität der laufenden Generatoren ausreichen, um eine Lasterhöhung von $\alpha_{oben} = 20$ über den geplanten Bedarf hinaus abzudecken, ohne dass neue Generatoren angelassen werden müssen und ohne dass die Maximallast eines Generators überschritten wird.

$$\sum_{g \in G} p_{max,g} \cdot u_{g,s,t} \geq (1 + \alpha_{oben}) \cdot d_{s,t} \qquad \text{für alle } s \in S,\ t \in T$$

6) Unterer Lastgangpuffer: Bei einer Lastreduktion von $\alpha_{unten} = 15$ darf keiner der laufenden Generatoren unter seine Minimallast fallen. Die Summe aller Minimallasten der laufenden Generatoren darf daher den reduzierten Bedarf nicht überschreiten.

$$\sum_{g \in G} p_{min,g} \cdot u_{g,s,t} \leq (1 - \alpha_{unten}) \cdot d_{s,t} \qquad \text{für alle } s \in S,\ t \in T$$

7-8) Netzwerkstabilität: Aus Gründen der Netzwerkstabilität dürfen nicht mehr als 3 Generatoren vom Typ 2 laufen, wenn währenddessen mehr als 3 Generatoren von Typ 1 verwendet werden. Diese Bedingung lässt sich nicht linear modellieren und wird deshalb mit der Binärvariablen $z_{s,t}$ und der Big-M-Methode modelliert. Die Big-M-Werte werden so eng wie möglich gewählt: $M_1 = n_1 - 3 = 3$ und $M_2 = n_2 - 3 = 5$. Big-M wird hier durch die Anzahl maximal verfügbarer Generatoren eingeschränkt.

Wenn $z_{s,t} = 0$: NB (7) erzwingt $u_{1,s,t} \leq 3$, NB (8) ist für Typ 2 inaktiv ($u_{2,s,t} \leq 8$). Wenn $z_{s,t} = 1$: NB (7) ist für Typ 1 inaktiv ($u_{1,s,t} \leq 6$), NB (8) erzwingt $u_{2,s,t} \leq 3$. In beiden Fällen ist sichergestellt, dass nie mehr als 3 Generatoren beider Typen gleichzeitig laufen.

$$u_{1,s,t} \leq 3 + (n_1 - 3) \cdot z_{s,t} \qquad \text{für alle } s \in S,\ t \in T$$

$$u_{2,s,t} \leq 3 + (n_2 - 3) \cdot \left(1 - z_{s,t}\right) \qquad \text{für alle } s \in S,\ t \in T$$

<u>9-11) Anlasskosten und Periodizität:</u> Ein Generator gilt als neu gestartet, wenn er im aktuellen Zeitschritt läuft, im vorherigen jedoch nicht. Da die Anlassvariable $v_{g,s,t}$ in der Zielfunktion minimiert wird, nimmt sie stets den kleinstmöglichen zulässigen Wert an, nämlich $\max\left(0,\ u_{g,s,t} - u_{g,s-1,t}\right)$.

Drei Übergangsfälle werden separat behandelt:

- der zyklische Wochenübergang von Sonntag auf Montag (NB 9)

$$v_{g,1,1} \geq u_{g,1,1} - u_{g,|S|,|T|} \qquad \text{für alle } g \in G$$

- der Stundenwechsel innerhalb eines Tages (NB 10)

$$v_{g,s,t} \geq u_{g,s,t} - u_{g,s-1,t} \qquad \text{für alle } g \in G,\ s \in S \setminus \{1\},\ t \in T$$

- sowie der Tageswechsel zur ersten Stunde eines neuen Tages (NB 11).

$$v_{g,1,t} \geq u_{g,1,t} - u_{g,|S|,t-1} \qquad \text{für alle } g \in G,\ t \in T \setminus \{1\}$$

<u>12) Nichtnegativitäts- und Ganzzahligkeitsbedingungen:</u> Alle Variablen sind nichtnegativ. $u_{g,s,t}$ ist zusätzlich ganzzahlig, $z_{s,t}$ binär.

$x_{g,s,t} \geq 0$ für alle $g \in G,\ s \in S,\ t \in T$

$u_{g,s,t} \geq 0$, ganzzahlig, für alle $g \in G,\ s \in S,\ t \in T$

$v_{g,s,t} \geq 0$ für alle $g \in G,\ s \in S,\ t \in T$

$z_{s,t} \in \{0,1\}$ für alle $s \in S,\ t \in T$

Anzahl Variablen & Nebenbedingungen

Die hier angegebenen Werte ergeben sich, wenn $|G| = 2$ (zwei Generatortypen), $|S| = 24$ (24 Stunden pro Tag) und $|T| = 7$ (sieben Tage pro Woche) sind. Für die zweite Fragestellung, ob zwei Generatoren des Typen 2 aufgewertet werden sollen, ergeben sich entsprechend andere Werte, da hier $|G| = 3$ gilt.

Insgesamt umfasst das Modell 1.176 Variablen und 2.184 Nebenbedingungen.

Variable	Typ	Anzahl
$\boldsymbol{x_{g,s,t}}$	kontinuierlich	$2 \times 24 \times 7 = 336$
$\boldsymbol{u_{g,s,t}}$	ganzzahlig	$2 \times 24 \times 7 = 336$
$\boldsymbol{v_{g,s,t}}$	kontinuierlich	$2 \times 24 \times 7 = 336$
$\boldsymbol{z_{s,t}}$	binär	$24 \times 7 = 168$
Gesamt		**1.176**

NB	Beschreibung	Anzahl
1	Bedarfsdeckung	$24 \times 7 = 168$
2	Produktionsminimum	$2 \times 24 \times 7 = 336$
3	Produktionsmaximum	$2 \times 24 \times 7 = 336$
4	Verfügbare Generatoren	$2 \times 24 \times 7 = 336$
5	Oberer Puffer	$24 \times 7 = 168$
6	Unterer Puffer	$24 \times 7 = 168$
7	Netzwerkstabilität Typ 1	$24 \times 7 = 168$
8	Netzwerkstabilität Typ 2	$24 \times 7 = 168$
9	Anlasskosten Wochenanfang	2
10	Anlasskosten Stundenwechsel	$2 \times 23 \times 7 = 322$
11	Anlasskosten Tageswechsel	$2 \times 6 = 12$
Gesamt		**2.184**

Anhang B – Python-Implementierung

Folgende Tabelle zeigt, wie die Modellgrößen in Python repräsentiert werden:

Sachverhalt in der Realität	Objekt im mathematischen Modell	Repräsentation in Python (_mod.py)
Mengen (Generatortypen, Stunden des Tages, Wochentage)	G, S, T	dat.GEN, at.STUNDE, dat.TAG
Oberer Lastgangpuffer	α_{oben}	dat.puffer_oben
Unterer Lastgangpuffer	α_{unten}	dat.puffer_unten
Anlasskosten je Generatortyp	$\mathrm{k}_{\mathrm{g}}^{\mathrm{A}}$	dat.anlassko[g]
Fixkosten auf Minimallast je Generatortyp	$\mathrm{k}_{\mathrm{g}}^{\mathrm{F}}$	dat.minlastko[g]
Variable Kosten oberhalb Minimallast	$\mathrm{k}_{\mathrm{g}}^{\mathrm{V}}$	dat.kouebermin[g]
Minimallast je Generatortyp	$\mathrm{p}_{min,g}$	dat.minlast[g]
Maximallast je Generatortyp	$p_{max,g}$	dat.maxlast[g]
Anzahl verfügbarer Generatoren je Typ	n_g	dat.verfgen[g]
Stündlicher Lastgang-Bedarf	$d_{s,t}$	dat.bedarf[s, t]
Gesamtproduktion je Typ, Stunde, Tag	$x_{g,s,t}$	ProdV[g, s, t]
Anzahl laufender Generatoren je Typ, Stunde, Tag	$u_{g,s,t}$	LaufGV[g, s, t]
Anzahl Neustarts je Typ, Stunde, Tag	$v_{g,s,t}$	NeuGV[g, s, t]
Netzwerkstabilitäts-Indikator	$z_{s,t}$	NetzV[s, t]

Zur Bestimmung der Kostenminimalen Stromerzeugung kommen die Python Skripte „*Abschlussprojekt_dat.py*“ und „*Abschlussprojekt_mod.py*“ sowie die Excel-Dateien „*ProjektLastgang.xlsx*“ und „*ProjektErgebnis.xlsx*“ zum Einsatz. Für die Betrachtung das Upgrade-Szenario wird dasselbe Prinzip angewendet: „*UpgradefrageAbschlussprojekt_dat.py*“, „*UpgradefrageAbschlussprojekt_mod.py*“, „*ProjektLastgangUpgrade.xlsx*“ und „*ProjektErgebnisUpgrade.xlsx*“. In beiden Fällen liest die _dat-Datei alle Eingangsdaten aus der jeweiligen Lastgang-Excel aus, während die _mod-Datei das Modell löst und die Ergebnisse in die entsprechende Ergebnis-Excel schreibt.

Hinweis zur Ergebnisdatei: Da im Kurs kein Befehl zur automatischen Erstellung von Excel-Dateien behandelt wurde, muss die jeweilige Ausgabe-Excel als Datei bereits im selben Ordner vorhanden und geschlossen sein, bevor das Modellskript ausgeführt wird. Das Skript schreibt dann die Ergebnisse direkt in diese Datei.

Anhang C – Beantwortung der Business-relevanten Fragen

Kostenminimale Stromerzeugung

Zur Beantwortung der Anforderung, wie kostenminimal Strom unter den Nebenbedingungen erzeugt werden kann, wurde das in Anhang A beschriebene mathematische Modell aufgestellt und wie in Anhang B erläutert in Python mittels Xpress programmiert. Das Ausführen des Modells liefert ein für das gegebene Szenario optimalen Stromerzeugungsplan. Die Gesamtkosten belaufen sich auf 1.437.135 €, mit folgender Kostenstruktur:

Kostenart	**Generator 1**	**Generator 2**	**Gesamt**	Anteil
Anlasskosten	52.800 €	81.000 €	133.800 €	9,3 %
Fixkosten Minimallast	144.000 €	444.600 €	588.600 €	41,0 %
Variable Kosten	56.047 €	658.688 €	714.735 €	49,7 %
Gesamt	252.847 €	1.184.288 €	1.437.135 €	100%
Anteil	17,6 %	82,4%	100%	

Zudem wird in der gleichen Excel der konkrete Plan ausgegeben, wann wie viele Generatoren welchen Typs aktiv sind und wie viel Strom jeweils produziert wird (s. Anhang G).

Auffällig ist, dass der überwiegende Anteil der Kosten durch den Generatortyp 2 entstehen. Daraus lässt sich schließen, dass die niedrigeren variablen Kosten der Typ-2 Generatoren die höheren Minimallastkosten mehr als ausgleichen und diese entsprechend kosteneffizienter arbeiten, falls der Bedarf hoch ist. Das kann auch aus Anhang G entnommen werden, während in sehr bedarfsarmen Perioden oft ausschließlich Typ-1 Generatoren laufen, laufen zu Hochlastzeiten teilweise sogar sowohl 8x Typ-2 Generatoren als auch 3x Typ-1 Generatoren (mehr ist aufgrund des Netzstabilitätskriteriums hier nicht möglich) gleichzeitig. Deswegen liegt die Vermutung nah, dass der Neuerwerb von Typ-2 Generatoren potenziell Kosten einsparen könnte.

Diese Vermutung haben wir überprüft und ermittelt, dass der Neuerwerb eines weiteren Typ-2 Generators etwa 4000 Euro pro Woche sparen würde (1.433.035 € Gesamtkosten). Je nach Anschaffungskosten und Laufzeit dieser Maschine könnte also eine Investition wirtschaftlich sein.

Die Betrachtung solch verschiedener Szenarien wird durch die Struktur unseres Modells möglich. Dafür muss die „ProjektLastgang.xlsx“ nach den entsprechenden Vorstellungen bearbeitet werden und das Modell erneut ausgeführt werden.

Maschinenaufwertung

Eine zentrale Frage, welche es im Rahmen der Untersuchung zu beantworten galt, war, ob ein Upgrade der Maximalkapazität von zwei Typ-2 Maschinen kostentechnisch sinnvoll wäre. Um diese Frage zu beantworten haben wir zwei Szenarien verglichen. Zum einen das bereits ermittelte Szenario zur kostenminimalen Stromerzeugung und ein neues, in welchem die verbesserten Maschinen integriert wurden. Dadurch wollten wir ermitteln, wie viel Kostenersparnis potenziell möglich wäre, um zu schätzen, wie hoch die Aufwertungskosten maximal sein sollten.

Dafür haben wir in der Excel eine weitere Maschinenkategorie hinzugefügt, welche bis auf die Maximallast identisch zur den bisherigen Typ-2 Maschinen ist. Die Maximallast haben wir deutlich erhöht auf 100.000 MW. Dadurch soll sichergestellt werden, dass falls die Maximalkapazität der bisherigen Typ-2 Maschinen bisher ein Bottleneck dargestellt haben, dass dieses Bottleneck nichtmehr existiert. Von diesen neuen Typ-2 Maschinen haben wir dem Modell zum Errechnen zwei zur Verfügung gestellt und entsprechend die Anzahl der alten Typ-2 Maschinen von 8 auf 6 verringert.

Daraufhin haben wir das Modell erneut ausgeführt. Dabei war die Lösung in Bezug auf die Gesamtkosten identisch. Daraus folgt, dass eine Aufwertung der Maximalkapazität zweier Typ-2 Maschinen, (selbst wenn diese kostenlos wäre) keine Einsparungen in Bezug auf die wöchentlichen Gesamtkosten hervorruft. Dementsprechend war eine weitere Untersuchung zur Beantwortung des Investitionsgesuchs nicht notwendig. Auch diese Excel ist beigefügt, sodass bei Bedarf weitere Abänderungen der bestehenden Maschinen, bspw. in Bezug auf variable Kosten geprüft werden könnten. Dadurch kann ermittelt werden, welche Ersparnis maximal möglich und entsprechend abgeleitet, welcher finanzielle Aufwand für die Verbesserung wirtschaftlich wäre.

Anhang D – Python Code

Hinweise: Der Python Code liegt auch ausführbar in Form von vier Python Dateien (.py) vor. Je eine Datendatei und eine Modelldatei für das reguläre Szenario und das Upgrade Szenario. Zudem wird die Library „useful_functions.py“ beigelegt, deren Methoden Verwendung finden. Wie in Anhang B erläutert, werden für das Upgrade Szenario nahezu identische Python Dateien (Daten & Modell) verwendet. Deshalb wurde diesem Bericht lediglich die Version für das reguläre Szenario angefügt. Die zwei Unterschiede zwischen Regulär und Upgrade bestehen in Zeile 31 der Datendatei: Hier wird aus 2 Generatortypen 3 und in Zeile 9 der Modell Datei, bei der nun statt der regulären Datendatei, die upgrade Version genutzt wird.

Abschlussprojekt_dat.py

```
 1. """
 2. Abschlussprojekt
 3. Daten-Datei
 4. @author: Antonia Schikora, Cedric Schuster, Jonas Stassiuk, Pascal Moroschan
 5. """
 6.
 7. import useful_functions as use
 8.
 9. # Öffnet das erste Tabellenblatt (Index 0) der Excel-Datei zum Lesen und Schreiben
10. workbook, sheet = use.open_sheet("ProjektLastgang.xlsx", 0)
11.
12. ################################
13. ### PARAMETER, MENGEN        ###
14. ################################
15.
16. ### Skalarparameter ###########
17.
18. # Lastgangpuffer, wie viel Spielraum ist nach oben und nach unten gegeben
19. puffer_oben = use.read_scalar(sheet, "L14")
20. puffer_unten = use.read_scalar(sheet, "L15")
21.
22. ### Mengen #####################
23.
24. # Wochentag
25. TAG = use.read_index(sheet, "B2", "horizontal", 7)
26.
27. # Tageszeit
28. STUNDE = use.read_index(sheet, "A3", "vertical", 24)
29.
30. # Generatortypen -> hier Zahl ändern, wenn mehr Generatortypen verwendet werden sollen
31. GEN = use.read_index(sheet, "K3", "vertical", 2)
32.
33. ### Vektorparameter ############
34.
35. # Anlasskosten (EUR pro Anlassung)
36. anlassko = use.read_table(sheet, "L3", GEN)
37.
38. # Fixkosten auf Minimallast (EUR pro Stunde je Generator)
39. minlastko = use.read_table(sheet, "O3", GEN)
40.
41. # Variable Kosten oberhalb Minimallast (EUR pro MW und Stunde)
42. kouebermin = use.read_table(sheet, "R3", GEN)
43.
44. # Minimallast (MW)
45. minlast = use.read_table(sheet, "U3", GEN)
```

```python

# Maximallast (MW)
maxlast = use.read_table(sheet, "X3", GEN)

# Anzahl verfügbarer Generatoren
verfgen = use.read_table(sheet, "AA3", GEN)

# Lastgang-Bedarf (MW)
bedarf = use.read_table(sheet, "B3", STUNDE, TAG)

```

Abschlussprojekt_mod.py

```python
"""
Abschlussprojekt
Modell-Datei
@author: Antonia Schikora, Cedric Schuster, Jonas Stassiuk, Pascal Moroschan
"""

import xpress as xp
import useful_functions as use
import Abschlussprojekt_dat as dat

lastgang = xp.problem()

################################
### VARIABLEN                ###
################################

# Gesamtproduktion aller Generatoren von Typ g in Stunde s an Tag t (MW)
ProdV = lastgang.addVariables(dat.GEN, dat.STUNDE, dat.TAG,
                              name="ProdV", lb=0, vartype=xp.continuous)

# Anzahl laufender Generatoren von Typ g in Stunde s an Tag t (ganzzahlig)
LaufGV = lastgang.addVariables(dat.GEN, dat.STUNDE, dat.TAG,
                               name="LaufGV", lb=0, vartype=xp.integer)

# Begründung für continuous: NeuGV erbt Ganzzahligkeit von LaufGV (da NeuGV >= LaufGV[t] - LaufGV[t-1],
# und LaufGV ganzzahlig ist, nimmt NeuGV im Optimum automatisch ganzzahlige Werte an)
NeuGV = lastgang.addVariables(dat.GEN, dat.STUNDE, dat.TAG,
                              name="NeuGV", lb=0, vartype=xp.continuous)

# Binärvariable für Netzwerkstabilität
# NetzV[s,t] = 1, wenn mehr als 3 Generatoren von Typ 1 laufen
NetzV = lastgang.addVariables(dat.STUNDE, dat.TAG,
                              name="NetzV", lb=0, vartype=xp.binary)

################################
### ZIELFUNKTION             ###
################################

gesamtkosten = xp.Sum(
    dat.anlassko[g]   * NeuGV[g, s, t]                                          # Anlasskosten
    + dat.minlastko[g]  * LaufGV[g, s, t]                                       # Fixkosten auf Minimallast
    + dat.kouebermin[g] * (ProdV[g, s, t] - dat.minlast[g] * LaufGV[g, s, t])  # Variable Kosten darüber
    for g in dat.GEN for s in dat.STUNDE for t in dat.TAG)

lastgang.setObjective(gesamtkosten, sense=xp.minimize)

################################
### NEBENBEDINGUNGEN         ###
################################

# Bedarfsdeckung (>= erlaubt minimale Überproduktion, die ZF verhindert sie automatisch)
bedarfsdeckung = [xp.Sum(ProdV[g, s, t] for g in dat.GEN) >= dat.bedarf[s, t]
```

```
                    for s in dat.STUNDE for t in dat.TAG]

# Minimalproduktion
produktionsminimum = [ProdV[g, s, t] >= dat.minlast[g] * LaufGV[g, s, t]
                      for g in dat.GEN for s in dat.STUNDE for t in dat.TAG]

# Maximalproduktion
produktionsmaximum = [ProdV[g, s, t] <= dat.maxlast[g] * LaufGV[g, s, t]
                      for g in dat.GEN for s in dat.STUNDE for t in dat.TAG]

# Obergrenze für Anzahl laufender Generatoren.
# Ohne diese NB könnte der Solver theoretisch mehr Generatoren einsetzen als verfügbar
maxAnzGen = [LaufGV[g, s, t] <= dat.verfgen[g]
             for g in dat.GEN for s in dat.STUNDE for t in dat.TAG]

# Oberer Puffer: laufende Generatoren müssen 20% Mehrbelastung ohne Neustart abdecken
obererPuffer = [xp.Sum(dat.maxlast[g] * LaufGV[g, s, t] for g in dat.GEN) >= (1 + dat.puffer_oben) * dat.bedarf[s, t]
                for s in dat.STUNDE for t in dat.TAG]

# Unterer Puffer: Minimallast darf bei 15% Rückgang nicht unterschritten werden
untererPuffer = [xp.Sum(dat.minlast[g] * LaufGV[g, s, t] for g in dat.GEN) <= (1 - dat.puffer_unten) * dat.bedarf[s, t]
                 for s in dat.STUNDE for t in dat.TAG]

# Netzwerkstabilität: nicht mehr als 3 Typ-1 UND mehr als 3 Typ-2 gleichzeitig
# Hinweis: GEN[0] = Typ 1, GEN[1] = Typ 2 — Reihenfolge muss der Excel entsprechen
M1 = dat.verfgen[dat.GEN[0]] - 3  # entspricht hier 3
M2 = dat.verfgen[dat.GEN[1]] - 3  # entspricht hier 5

netzwerkstabilitaet1 = [LaufGV[dat.GEN[0], s, t] <= 3 + M1 * NetzV[s, t]
                        for s in dat.STUNDE for t in dat.TAG]
netzwerkstabilitaet2 = [LaufGV[dat.GEN[1], s, t] <= 3 + M2 * (1 - NetzV[s, t])
                        for s in dat.STUNDE for t in dat.TAG]

# Anlasskosten: Woche ist periodisch
# Übergang Sonntag 23:00 auf Montag 0:00: gleiche Generatoren = keine Anlasskosten
# STUNDE[-1] = letzte Stunde, TAG[-1] = Sonntag
neugestartet_wochenanfang = [
    NeuGV[g, dat.STUNDE[0], dat.TAG[0]] >= LaufGV[g, dat.STUNDE[0], dat.TAG[0]] - LaufGV[g, dat.STUNDE[-1], dat.TAG[-1]]
    for g in dat.GEN]

neugestartet_stunde = [
    NeuGV[g, dat.STUNDE[s], t] >= LaufGV[g, dat.STUNDE[s], t] - LaufGV[g, dat.STUNDE[s-1], t]
    for g in dat.GEN for s in range(1, len(dat.STUNDE)) for t in dat.TAG]

neugestartet_tag = [
    NeuGV[g, dat.STUNDE[0], dat.TAG[t]] >= LaufGV[g, dat.STUNDE[0], dat.TAG[t]] - LaufGV[g, dat.STUNDE[0], dat.TAG[t-1]]
    for g in dat.GEN for t in range(1, len(dat.TAG))]

# Alle Nebenbedingungen hinzufügen
lastgang.addConstraint(
    bedarfsdeckung, produktionsminimum, produktionsmaximum, maxAnzGen,
    obererPuffer, untererPuffer,
    netzwerkstabilitaet1, netzwerkstabilitaet2,
    neugestartet_wochenanfang, neugestartet_stunde, neugestartet_tag)

################################
### INSTANZ LOESEN & AUSGABE ###
################################

lastgang.mipOptimize()

# Loesung anfordern
prod_plan  = lastgang.getSolution(ProdV)
laufend    = lastgang.getSolution(LaufGV)
neugestart = lastgang.getSolution(NeuGV)
gesamtkosten_wert = lastgang.attributes.objval
```

```python

# Konsolenausgabe
use.output("Produktion (MW)", prod_plan, dat.GEN, dat.STUNDE, dat.TAG)
use.output("Laufende Generatoren", laufend, dat.GEN, dat.STUNDE, dat.TAG)
use.output("Neu gestartete Generatoren", neugestart, dat.GEN, dat.STUNDE, dat.TAG)
print("\nGesamtkosten:", gesamtkosten_wert, "EUR")

################################
### EXCEL-AUSGABE            ###
################################

# HINWEIS: Vor dem ersten Ausführen dieses Skripts muss im selben Ordner
# eine leere Excel-Datei mit dem Namen "ProjektErgebnis.xlsx" erstellt werden.
print("HINWEIS: Vor dem ersten Ausführen dieses Skripts muss im selben Ordner eine leere Excel-Datei mit dem Namen 'ProjektErgebnis.xlsx' erstellt werden")
# Dazu einfach eine neue Excel-Datei anlegen, speichern und wieder schließen.
# Die Datei muss beim Ausführen des Skripts geschlossen sein.
workbook_out, sheet_out = use.open_sheet("ProjektErgebnis.xlsx", 0)

# Produktionsplan: Gesamtproduktion je Generatortyp, Stunde und Tag (MW).
use.write_thead(sheet_out, "A1", "Produktion (MW)", "Generator", "Stunde", "Wochentag")
use.write_tbody(sheet_out, "A1", prod_plan, dat.GEN, dat.STUNDE, dat.TAG)

# Anzahl laufender Generatoren je Typ, Stunde und Tag (ganzzahlig).
use.write_thead(sheet_out, "K1", "Laufende Generatoren", "Generator", "Stunde", "Wochentag")
use.write_tbody(sheet_out, "K1", laufend, dat.GEN, dat.STUNDE, dat.TAG)

# Kostenaufschlüsselung

# Anlasskosten je Generatortyp (EUR):
# anlassko[g] × Summe aller Neustarts über alle Stunden und Tage
anlasskosten_gen = {g: sum(dat.anlassko[g] * neugestart[g, s, t]
                           for s in dat.STUNDE for t in dat.TAG)
                    for g in dat.GEN}

# Fixkosten auf Minimallast je Generatortyp (EUR):
# minlastko[g] × Summe aller Laufstunden
fixkosten_gen = {g: sum(dat.minlastko[g] * laufend[g, s, t]
                        for s in dat.STUNDE for t in dat.TAG)
                 for g in dat.GEN}

# Variable Kosten oberhalb der Minimallast je Generatortyp (EUR):
# kouebermin[g] × (Gesamtproduktion – Minimallast × Anzahl laufender Generatoren)
varkosten_gen = {g: sum(dat.kouebermin[g] * (prod_plan[g, s, t] - dat.minlast[g] * laufend[g, s, t])
                        for s in dat.STUNDE for t in dat.TAG)
                 for g in dat.GEN}

# Ausgabe der drei Kostenkomponenten
use.write_thead(sheet_out, "U1", "Anlasskosten (EUR)", "Generator")
use.write_tbody(sheet_out, "U1", anlasskosten_gen, dat.GEN)

use.write_thead(sheet_out, "U5", "Fixkosten auf Minimallast (EUR)", "Generator")
use.write_tbody(sheet_out, "U5", fixkosten_gen, dat.GEN)

use.write_thead(sheet_out, "U9", "Variable Kosten oberhalb Minimallast (EUR)", "Generator")
use.write_tbody(sheet_out, "U9", varkosten_gen, dat.GEN)

# Gesamtkosten (= Zielfunktionswert)
use.write_scalar(sheet_out, "U13", gesamtkosten_wert, "Gesamtkosten (EUR)")

# Ergebnisse in der Ausgabedatei speichern.
use.save_sheet(workbook_out, "ProjektErgebnis.xlsx")

```

Anhang F – Modifizierte Daten Excel für das Upgrade Szenario

Hinweis: Die bereits im Rahmen der Aufgabenstellung gegebene Excel mit dem Namen „ProjektLastgang.xlsx“ ist ausschließlich als .xlsx Datei beigelegt und nicht diesem Bericht angefügt.

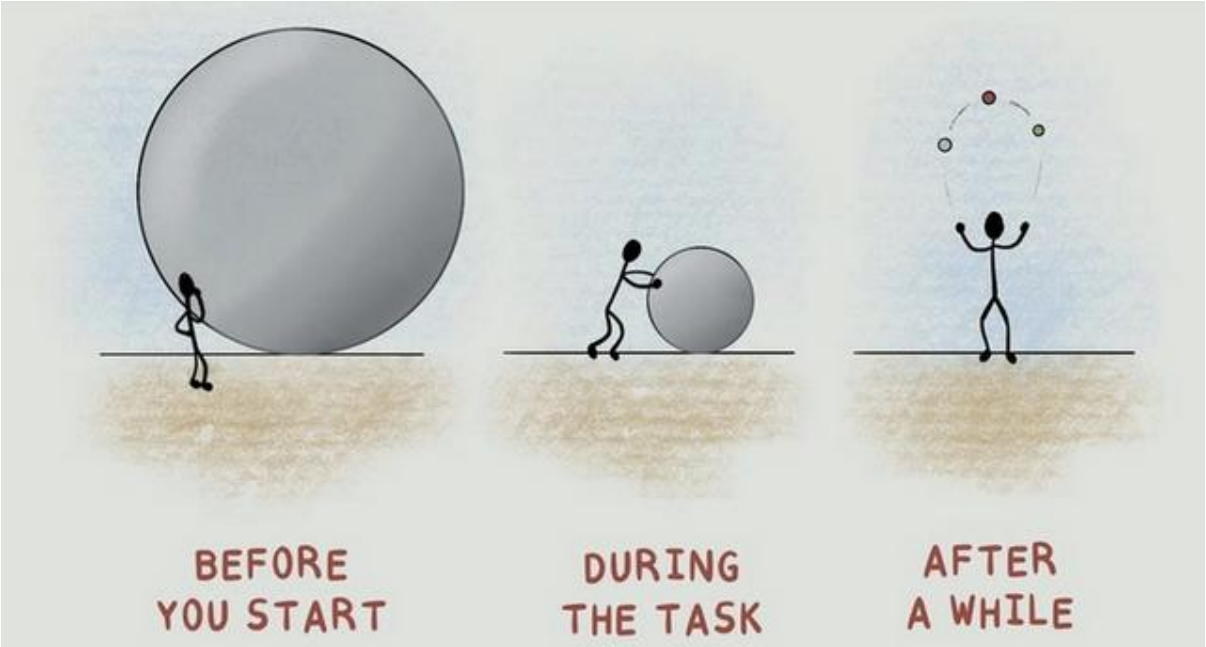
Anhang G – Excel Ergebnisse Teil 1: Reguläres Szenario

Anhang H – Excel Ergebnisse Teil 2: Upgrade Szenario

Aufgabenverteilung und ehrenwörtliche Erklärung

Die Aufgaben wurden nach unserem Empfinden gut und fair untereinander verteilt, wir haben auch deutlich über diese grobe Gliederung hinaus miteinander kollaboriert. Zum Beispiel wurden Ergebnisse von weiteren Gruppenmitgliedern validiert und bei Bedarf geholfen.

Aufgabe	**Gruppenmitglieder**
Management Summary	Jonas
Anhang A	Cedric
Anhang B	Pascal
Anhang C	Jonas
Modellierung Mathematisch	Gemeinsam
Umsetzung in Xpress	Antonia
Finale Abgabe & Formalien	Jonas, Pascal, Cedric



„Ich erkläre ehrenwörtlich:

dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe, und dass die Arbeit in gleicher oder ähnlicher Form in noch keiner anderen Prüfung vorgelegen hat. Sämtliche wörtlichen oder sinngemäßen Übernahmen und Zitate, sowie alle Abschnitte, die mithilfe von KI basierten Tools entworfen, verfasst und/oder bearbeitet wurden, sind kenntlich gemacht und nachgewiesen. Im Anhang meiner Arbeit habe ich sämtliche KI-basierte Hilfsmittel angegeben. Diese sind mit Produktnamen und formulierten Eingaben (Prompts) in einem KI-Verzeichnis ausgewiesen.

Ich bin mir bewusst, dass die Verwendung von Texten oder anderen Inhalten und Produkten, die durch KI-basierte Tools generiert wurden, keine Garantie für deren Qualität darstellt. Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.“

26.04.2026

Antonia Schikora

Cedric Schuster

Jonas Stassiuk

Pascal Moroschan